

CSC4201 — Mini-Project — Part 1

For this (small!) project, you are part of a team that is designing a simple ecommerce application. You are not responsible for the entire application (at least, not yet!), but you are responsible for one of the services.

The Services

Here we are documenting a minimal API for each of the services. For now you are only implementing one of them, but it may be helpful to see how your service fits into the larger whole.

Catalog service provides a listing of the products available on the site. Each product has a product id, name, textual description, price in US Dollars, and list of categories.

It provides one endpoint (`/products`) for returning a list of (all) items, with an optional search string to return only those items whose name or description contains the search string. It also provides an endpoint (`/products/product_id`) for each individual product.

Cart stores each currently-connected user's cart in a Redis DB. Users are assigned an ID by session, and do not log in.

It provides a single endpoint (`/cart/user_id`), with GET requests returning the cart contents, POST adding an item to the cart, and DELETE emptying the cart.

Payment Given credit-card information (card number, date, SVN, etc.), validate the card, then (mock) "charge" it.

It provides a single endpoint where we post charges, passing in both the amount charged, and the card information. It will return either an error (the card is invalid/expired/declined/etc.) or a transaction id (randomly-generated UUID for our purposes).

Shipping service both provides estimated shipping costs, and (mock) ships orders.

A GET request for `/shipping/order_id` should return the shipping estimate, while a POST of a valid shipping address triggers the (mock) shipping process.

Notification service (mock) sends an email confirming each order.

POST to `/emails/user_id` to send a message.

Checkout service will orchestrate other services to (mock) checkout a customer. This can be (mock) triggered by POSTing payment and shipping information to `/checkout/user_id`.

Recommendation service returns a list of up to 5 products from the catalog which are recommended based on the user's current cart contents.

It has a single endpoint `recommendations/user_id`

Frontend service will serve webpages whose content depends on the other services.

Endpoints without specified methods are assumed to use GET.

N.B. Many of the services are *mock* implemented. We are not actually running a store and do not wish to actually bill credit cards, ship goods, or send email.

Cart Service

You are in charge of implementing the **Cart** service. It should use a Redis database to store the contents of each customer's cart. (More than one simultaneous customer is possible.) Make sure that the database is not ephemeral; all instances of your service should share the same database. We will not (yet) verify user ids; presumably they are set/enforced by the frontend service.

Other Requirements

- You should create a **GitHub** repo for your service, and add me to it (so that I can at least see your code).
- Make a pipeline to unit-test your service before deploying it (and any dummy services or infrastructure that it uses) to the cloud.
- Demonstrate your service running in the cloud during or prior to next week's lab.
- Submit the URL of your repo to Canvas to indicate you are done with part 1.